ForgeWeb: Architecture-First Full-Stack Web Application Generation

Pratyush Manas, Rajeev Kumar, Sachin Yash Raj, Sanaa Kumari
Department of Computer Science and Engineering
BMS Institute of Technology and Management, Bengaluru, India
1BY23CS163 1BY23CS171 1BY23CS192 1BY23CS202

***Abstract*—ForgeWeb translates product descriptions into reviewed requirements and an approved architecture before coordinating source generation. The revised design retains Gemini for planning and React output and proposes a locally fine-tuned, pretrained LLaMA-based model for Python FastAPI generation. The workspace exposes files, previews, versions, and requirement links. Fourteen prior platform tests and type checking support the existing workflow, not the proposed model. Training data, checkpoint selection, and code-quality evaluation remain to be finalized. This paper separates implemented artifact management from proposed model adaptation and identifies execution-based testing as the next acceptance boundary.**

***Index Terms*—AI-assisted development, web application generation, local language models, fine-tuning, requirement traceability.**

I. Introduction

Turning a customer brief into software requires decisions about screens, data, permissions, and interactions. The result must represent the requested product: a restaurant needs a booking journey rather than a renamed dashboard, and an explicit color preference must affect the rendered interface.

Code-generation research demonstrates substantial progress on executable programming tasks. Codex evaluates generated functions through HumanEval [1]; APPS examines a broader collection of programming challenges [2]; and AlphaCode combines generation with selection mechanisms for competition problems [3]. Building an application introduces additional concerns: coordinating several files, matching frontend calls to backend routes, and allowing a person to correct the intended design.

ForgeWeb connects a prompt, an approved specification, generated artifacts, and recorded checks. The revised backend design adapts a pretrained LLaMA-based model for local code generation; it does not claim a newly pretrained foundation model. The platform's own interface remains separate from the customer's requested design. Local backend inference does not make the whole pipeline offline because planning and frontend generation still use Gemini.

II. Problem Statement and Objectives

Product descriptions combine features, preferences, and unstated assumptions. Generation may change a name while retaining an unsuitable layout, and an attractive preview may conceal missing server behavior. The problem is to preserve product intent while exposing enough evidence to assess the result. ForgeWeb creates an architecture file before implementation and supports revision, approval, source inspection, and version history. Its objectives are to respect explicit design choices, coordinate frontend and backend contracts, and separate artifact creation from runtime correctness.

III. Related Work

A. Generation and Evaluation

CodeGen explores multi-turn synthesis [4], while InCoder supports contextual code generation [5]. Code Llama demonstrates code-oriented adaptation of the Llama family [6], motivating a candidate local backbone without establishing which checkpoint suits this project. CodeT uses generated tests for selection [7]; EvalPlus strengthens correctness checks [8]; LiveCodeBench addresses multiple coding abilities and contamination [9]; and BigCodeBench evaluates complex library-based tasks [10]. Together these studies motivate evaluation beyond file presence.

B. Agent Workflows and Planning

SWE-bench evaluates repository issue resolution [11], and SWE-agent studies the interface between an agent and its development tools [12]. ChatDev [13], MetaGPT [14], and AutoGen [15] organize collaboration through roles or conversations. Agentless provides a useful counterpoint by examining a simpler repair pipeline [16]. ForgeWeb uses task roles to organize scope, but its current reviewer is a server function rather than an independently reasoning agent.

ReAct connects reasoning to environmental actions [17]. Reflexion [18] and Self-Refine [19] investigate feedback-driven improvement. Chain-of-thought [20] and Plan-and-Solve [21] provide further motivation for intermediate planning. ForgeWeb's architecture document is a user-facing agreement about the product; it is not presented as a reproduction of these reasoning methods.

C. Context, Appearance, and Safety

Retrieval-augmented generation grounds responses in retrieved material [22], and RepoCoder applies retrieval to repository completion [23]. Both suggest future support for edits spanning large projects. Design2Code examines visual frontend reconstruction [24], supporting the need to assess layout separately from syntax. Security studies by Pearce *et*

al. [25] and Perry *et al.* [26] show why generated security-related code should not automatically inspire confidence. These findings shape ForgeWeb's validation boundary rather than supply performance claims for the prototype.

IV. System Architecture

A. User Layer and Control Service

The user layer uses React and TypeScript. It contains the prompt composer, proposal review, project drawer, file viewer, preview, and version controls. A Node.js HTTP service coordinates these operations, with Vite middleware exposing the same handler during development. Workflow states distinguish planning, confirmation, generation, review, validation, and completion or failure. Figure 1 outlines this separation.

Project state is persisted in a local JSON store. Queued writes and temporary-file replacement reduce conflicts within the local process. This arrangement supports the prototype, but is not a substitute for a transactional multi-tenant database. Database information shown in the workspace describes intended application entities; it does not prove that a database has been provisioned.

B. Generation and Artifact Storage

The proposed split pipeline uses Gemini for planning and frontend generation and a locally served, fine-tuned LLaMA-based model for backend source. A local inference adapter would receive the approved architecture, entities, routes, and API contract, then return a validated file manifest. Backend generation would no longer depend on a hosted backend-code API. The local checkpoint, model size, hardware, and serving configuration are not yet fixed; no latency or privacy improvement is claimed without measurement.

Local generators cover selected commerce, restaurant, portal, inventory, scheduling, and generic-site cases. They provide a bounded fallback rather than unlimited design originality. Generator mode and failure messages remain attached to the result. Stored artifacts include source, a shared contract, documentation, preview HTML, and file digests. Snapshots support inspection, restoration, and ZIP export.

C. Proposed Model Adaptation

Training would pair architecture and API specifications with reviewed Python implementations. Data must be license-compatible, stripped of secrets, deduplicated, and split by project to reduce train–test leakage. A pretrained LLaMA-based checkpoint would be fine-tuned on these pairs and served locally after evaluation. The untouched base model and adapted model should be compared on held-out builds, API behavior, and authorization tests. Checkpoint identity, dataset provenance, training settings, and hardware must be recorded before reporting results.

V. Methodology

A. Planning and Approval

The planner turns a validated prompt into a proposed specification and `ARCHITECTURE.md`. Explicit names and visual preferences should survive normalization. The user can edit the request or confirm the proposal; implementation begins only after confirmation. This boundary checks agreement about scope, not the correctness of later code. Figure 2 shows the sequence and failure branch.

B. Frontend and Backend Coordination

The approved specification guides React source, styles, FastAPI routes, schemas, access-control files, and supporting tests. A shared JSON contract expresses the intended route and data agreement. Figure 3 separates these files from the preview path. Contract presence reduces ambiguity but cannot establish that implementations agree: a booking form and its endpoint can still interpret dates or identifiers differently.

The embedded preview runs stored HTML in an iframe with script and form permissions. It does not start the generated Python service. Local preview interactions must therefore be distinguished from verified frontend-to-backend requests. Running the exported application requires its dependencies and data services to be configured.

C. Review and Traceability

Planning, implementation, review, and validation records describe objectives, allowed paths, and acceptance criteria. Project policy draws on the supplied Karpathy-derived engineering guidance and gstack workflow material: clarify assumptions, prefer a sufficient solution, limit changes to scope, and inspect evidence. These engineering repositories are not counted as research publications.

The current checks examine paths, manifests, expected artifacts, requirement mappings, and provenance. The internal graph links projects, requirements, tasks, files, and checks. A requirement-to-file edge records an association, not proof that a feature works. Similarly, the graph is not an externally parsed call graph from Code Review Graph. Stronger acceptance would combine the current structural checks with clean builds, API requests, and browser interaction tests against the generated application.

VI. Evaluation and Discussion

The earlier platform evaluation passed fourteen top-level tests and TypeScript checking. Table I reports that baseline, not a rerun of a local-model integration. Temporary stores and deterministic or injected responses exercise orchestration; no fine-tuned checkpoint was tested. Local-model accuracy, runtime, and resource use remain unmeasured.

Approval checks enforce confirmation before implementation. Domain checks examine product-specific output and explicit colors, but not perceived uniqueness. Workspace checks cover snapshots, restoration, persistence, export, and safe failure. Provider fixtures verify plan and manifest handling, not live availability.

The results support the orchestration and artifact-management design. They do not establish secure deployment, payment correctness, or successful execution of every generated interaction. A useful next experiment would compare

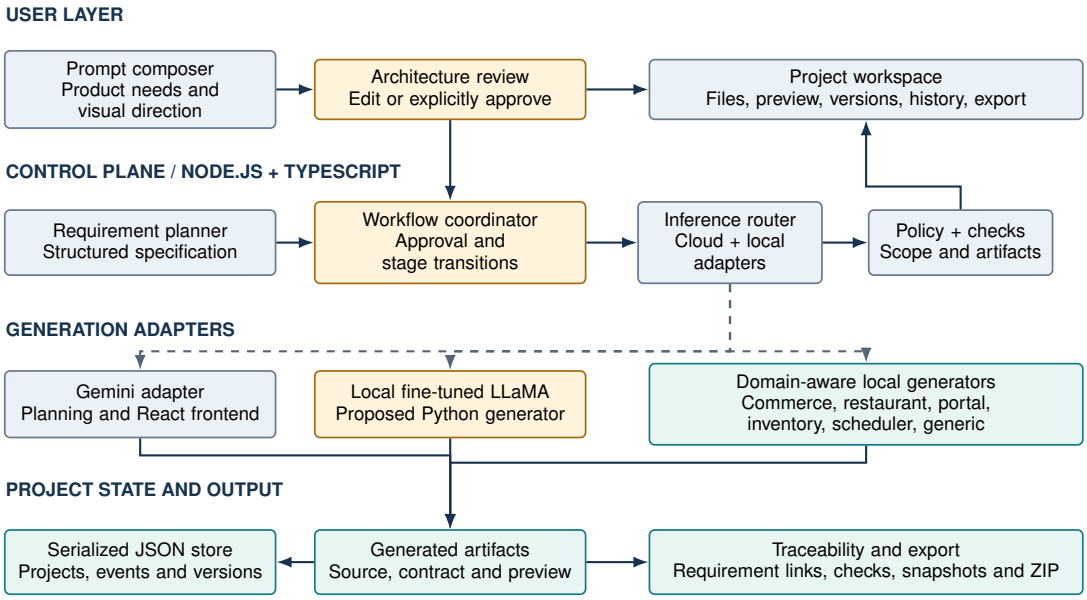


Fig. 1. Revised system architecture: the existing control plane with a proposed locally fine-tuned LLaMA backend generator. Gemini remains responsible for planning and frontend output. The trained checkpoint is not yet evaluated.

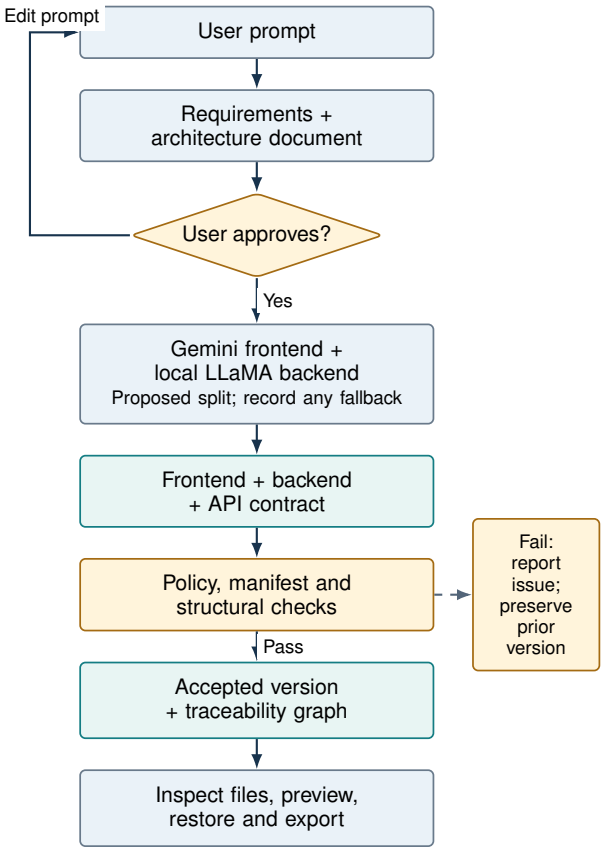


Fig. 2. Revised data flow with proposed local-model backend generation. A recorded template fallback is distinct from model inference.

direct generation with architecture-first generation on the same prompts and provider settings. Measurements should include

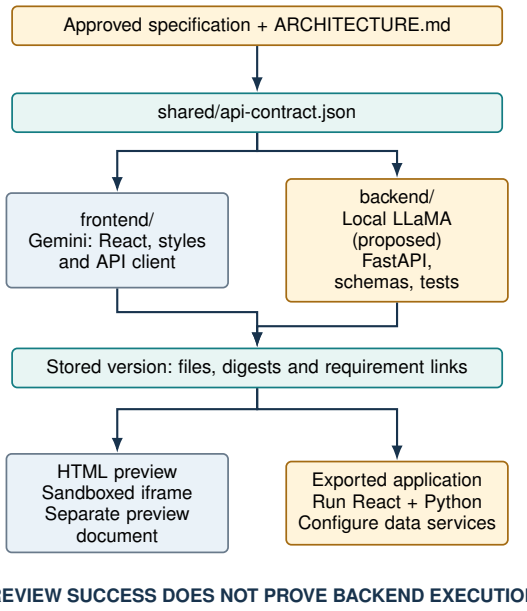


Fig. 3. Generated artifacts under the proposed split: Gemini frontend and locally fine-tuned LLaMA backend. Preview and application execution remain separate.

requirement-level task success, contract agreement, clean-build success, visual adherence, latency, and cost. Such comparative measurements have not been conducted here.

VII. Limitations and Future Work

Source checks can miss broken handlers, incorrect permissions, and mismatched payloads. Fallback designs remain limited, model responses vary, and local persistence is unsuitable for production-scale tenancy.

TABLE I
PRIOR PLATFORM CHECKS (NOT LOCAL-MODEL RESULTS)

Test group	Cases	Result
HTTP and approval lifecycle	2	Passed
Domain and visual instructions	4	Passed
Provider fixture and selection	2	Passed
Plan normalization	1	Passed
Workspace, versions, compatibility	3	Passed
Prompt and path rejection	2	Passed
Total	14	Passed
Platform type checking	—	Passed

The immediate priority is selecting, fine-tuning, and evaluating the local LLaMA-based checkpoint, then connecting its manifest output to the workflow. Isolated execution, persistent application data, and browser tests remain necessary. Training does not itself establish code correctness or security, and human assessment must verify visual requirements.

VIII. CONCLUSION

ForgeWeb makes planning and approval explicit before source generation. The revised design combines Gemini-assisted frontend generation with a proposed locally fine-tuned LLaMA backend model. Existing checks support the surrounding workflow, not the untested checkpoint. Training evidence and executed application tests are needed before assessing the revised pipeline's effectiveness.

REFERENCES

[1] M. Chen, J. Tworek, H. Jun *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[2] D. Hendrycks, S. Basart, S. Kadavath *et al.*, "Measuring coding challenge competence with APPS," *arXiv preprint arXiv:2105.09938*, 2021. [Online]. Available: https://arxiv.org/abs/2105.09938

[3] Y. Li, D. Choi, J. Chung *et al.*, "Competition-level code generation with AlphaCode," *arXiv preprint arXiv:2203.07814*, 2022. [Online]. Available: https://arxiv.org/abs/2203.07814

[4] E. Nijkamp, B. Pang, H. Hayashi *et al.*, "CodeGen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022. [Online]. Available: https://arxiv.org/abs/2203.13474

[5] D. Fried, A. Aghajanyan, J. Lin *et al.*, "InCoder: A generative model for code infilling and synthesis," in *Proc. International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2204.05999

[6] B. Rozière, J. Gehring, F. Gloeckle *et al.*, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023. [Online]. Available: https://arxiv.org/abs/2308.12950

[7] B. Chen, F. Zhang, A. Nguyen *et al.*, "CodeT: Code generation with generated tests," *arXiv preprint arXiv:2207.10397*, 2022. [Online]. Available: https://arxiv.org/abs/2207.10397

[8] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," *arXiv preprint arXiv:2305.01210*, 2023. [Online]. Available: https://arxiv.org/abs/2305.01210

[9] N. Jain, K. Han, A. Gu *et al.*, "LiveCodeBench: Holistic and contamination free evaluation of large language models for code," *arXiv preprint arXiv:2403.07974*, 2024. [Online]. Available: https://arxiv.org/abs/2403.07974

[10] T. Y. Zhuo, M. C. Vu, J. Chim *et al.*, "BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions," *arXiv preprint arXiv:2406.15877*, 2024. [Online]. Available: https://arxiv.org/abs/2406.15877

[11] C. E. Jimenez, J. Yang, A. Wettig *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.06770

[12] J. Yang, C. E. Jimenez, A. Wettig *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024. [Online]. Available: https://arxiv.org/abs/2405.15793

[13] C. Qian, W. Liu, H. Liu *et al.*, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 15 174–15 186. [Online]. Available: https://aclanthology.org/2024.acl-long.810/

[14] S. Hong, M. Zhuge, J. Chen *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, 2023. [Online]. Available: https://arxiv.org/abs/2308.00352

[15] Q. Wu, G. Bansal, J. Zhang *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[16] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," *arXiv preprint arXiv:2407.01489*, 2024. [Online]. Available: https://arxiv.org/abs/2407.01489

[17] S. Yao, J. Zhao, D. Yu *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[18] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *arXiv preprint arXiv:2303.11366*, 2023. [Online]. Available: https://arxiv.org/abs/2303.11366

[19] A. Madaan, N. Tandon, P. Gupta *et al.*, "Self-Refine: Iterative refinement with self-feedback," *arXiv preprint arXiv:2303.17651*, 2023. [Online]. Available: https://arxiv.org/abs/2303.17651

[20] J. Wei, X. Wang, D. Schuurmans *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *arXiv preprint arXiv:2201.11903*, 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[21] L. Wang, W. Xu, Y. Lan *et al.*, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proc. 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 2609–2634. [Online]. Available: https://aclanthology.org/2023.acl-long.147/

[22] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *arXiv preprint arXiv:2005.11401*, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[23] F. Zhang, B. Chen, Y. Zhang *et al.*, "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proc. 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2471–2484. [Online]. Available: https://aclanthology.org/2023.emnlp-main.151/

[24] C. Si, Y. Zhang, R. Li, Z. Yang, R. Liu, and D. Yang, "Design2Code: Benchmarking multimodal code generation for automated front-end engineering," in *Proc. 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025, pp. 3956–3974. [Online]. Available: https://aclanthology.org/2025.naacl-long.199/

[25] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symposium on Security and Privacy (SP)*, 2022. [Online]. Available: https://arxiv.org/abs/2108.09293

[26] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 2785–2799. [Online]. Available: https://arxiv.org/abs/2211.03622